T.C.

MARMARA ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

MİKROİŞLEMCİLER DERSİ (BLM2008)

DÖNEM PROJESİ RAPORU

REFLEKS OYUNU (NUMPAD HIZ TESTİ SİMÜLASYONU)

Geliştirme Ortamı: Emu8086 (8086 Mikroişlemci Emülatörü)

Proje Kaynak Kodları (GitHub): https://github.com/tunahankavakli/Numpad-Reflex-8086

HAZIRLAYAN:

Tunahan KAVAKLI – 170424506

Teslim Tarihi: 27.04.2026

İçindekiler

1. PROJENİN AMACI VE KAPSAMI.....3

2. KULLANILAN TEKNOLOJİLER VE ARAÇLAR.....3

3. SİSTEM MİMARİSİ VE KULLANILAN DONANIM KESMELERİ (INTERRUPTS).....3

4. ALGORİTMA VE OYUN AKIŞI.....4

1. Oyunun Başlaması ve Rastgele Sayı Üretimi.....4

2. Asenkron Bekleme ve Tuş Kontrolü.....4

3. Oyun Sonu ve İstatistik Gösterimi.....5

5. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER.....5

6. SONUÇ VE DEĞERLENDİRME.....6

1. PROJENİN AMACI VE KAPSAMI

Bu projenin amacı, x86 Assembly dili kullanılarak donanım seviyesinde (klavye tamponu ve sistem saati) asenkron işlemler yapabilen, kullanıcının tepki süresini ölçen bir refleks oyunu geliştirmektir.

Proje isterlerine uygun olarak sistem şu temel kurallara göre tasarlanmıştır:

- Oyuncu ekranda rastgele çıkan komut tuşlarına mümkün olan en kısa sürede basmaya çalışır.
- Komutlar arası bekleme süresi giderek kısalmalıdır.
- Yanlış tuş basını hata sayısını artırmalıdır.
- 3 hata yapan oyuncu oyundan elenmelidir.
- Ortalama tepki süresi hesaplanmalıdır.

Proje, bu kuralları standart klavye harfleri yerine "Numpad (Sayısal Tuş Takımı)" üzerinden 0-9 arası rakamlarla gerçekleştirerek, veri girişi hızını artırmaya yönelik pratik bir simülasyon haline getirilmiştir.

2. KULLANILAN TEKNOLOJİLER VE ARAÇLAR

- **Dil:** 8086 Assembly Dili (16-bit)
- **Derleyici/Emülatör:** Emu8086
- **Dosya Formatı:** .COM (100h bellek ofsetinden başlayan tek segmentli yürütülebilir dosya)

3. SİSTEM MİMARİSİ VE KULLANILAN DONANIM KESMELERİ (INTERRUPTS)

Assembly dilinin donanıma doğrudan erişim gücünden faydalanılarak projede aşağıdaki BIOS ve DOS kesmeleri aktif olarak kullanılmıştır:

- **Sistem Saati (INT 1Ah, AH=00h):** Bilgisayarın BIOS saatini "tick" (1 tick ≈ 55 ms) cinsinden okumak için kullanılmıştır.

- **Asenkron Klavye Okuma (INT 16h, AH=01h):** Programın akışını durdurmadan (non-blocking) klavye tamponunu (buffer) dinlemek için kullanılmıştır. Bu sayede program arka planda süreyi saymaya devam edebilmiştir.
- **Klavye Tamponu Temizliği (INT 21h, AH=0Ch):** Süre aşımında veya ardışık hatalı basımlarda, klavye belleğinde biriken tuş vuruşlarını temizleyerek oyunun kilitlenmesi engellenmiştir.

4. ALGORİTMA VE OYUN AKIŞI

1. Oyunun Başlaması ve Rastgele Sayı Üretimi

Program açıldığında kullanıcıyı bilgilendiren bir arayüz karşılar ve ENTER tuşuna basılması beklenir. Oyun başladığında sistem saati üzerinden mod 10 işlemi yapılarak 0-9 aralığında rastgele rakamlar üretilir.

Resim 1: *Oyunun başlangıç ekranı, kullanıcıyı karşılayan arayüz ve kurallar.*

```
; 1. Rastgele Rakam Secimi (0-9 arasi)
mov ah, 00h
int 1Ah
mov ax, dx
mov dx, 0
mov bx, 10
div bx

mov bx, dx
mov al, harfler[bx]
mov hedef_tus, al
```

Resim 2: *Sistem saati kesmesi (INT 1Ah) kullanılarak rastgele hedef rakamın üretildiği kod bloğu.*

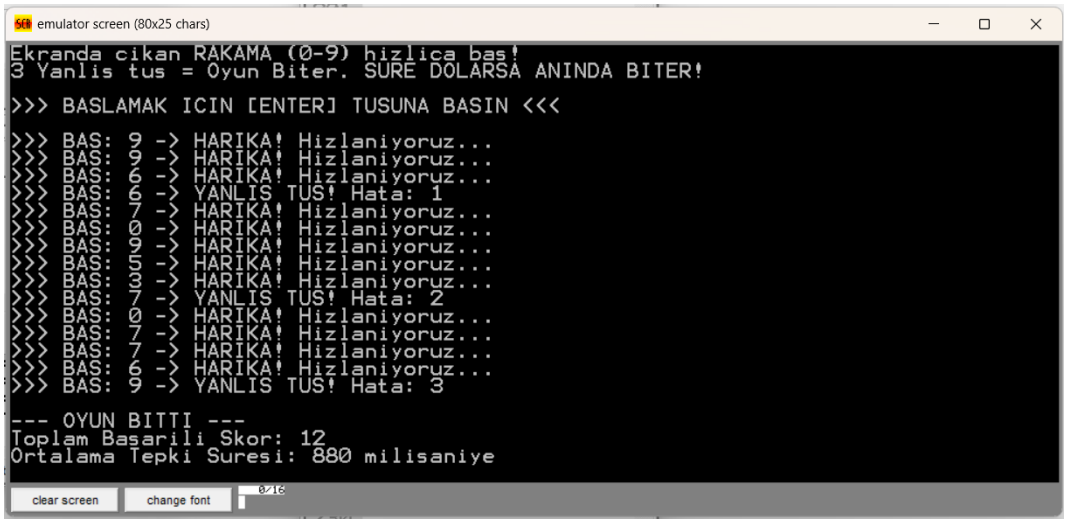
2. Asenkron Bekleme ve Tuş Kontrolü

Program sürekli olarak saati ve klavyeyi kontrol eden bir döngüye girer. Eğer geçen süre, anlık zaman limitini aşarsa oyuncu süre aşımından dolayı doğrudan kaybeder. Tuşa basıldığında anlık süre kaydedilir. Basılan tuş hedef ile uyuşuyorsa oyunun zorluk limitleri güncellenir (`zaman_limit` 3 tick daraltılır).

Resim 3: *Oyunun aktif oynanış anı ve ekranda beliren hedef rakam gösterimi.*

3. Oyun Sonu ve İstatistik Gösterimi

Oyun bittiğinde, oyuncunun harcadığı toplam süre (tick cinsinden) başarılı atış sayısına bölünür. Elde edilen değer 55 ile çarpılarak "Milisaniye" formatında ekrana yansıtılır.



Resim 4: *Oyun sonu ekranında hesaplanan toplam skor ve ortalama tepki süresi (milisaniye) istatistikleri.*

5. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER

- **Zorluk:** Assembly dilinde yerleşik bir "random" veya "delay" fonksiyonunun olmaması.

- **Çözüm:** BIOS saat kesmesi (`INT 1Ah`) manipüle edilerek hem rastgelelik hem de asenkron bekleme mekanizmaları manuel olarak kurgulanmıştır.

- **Zorluk:** İki haneli veya daha büyük sayıların (örneğin 550 milisaniye) ekrana yazdırılamaması (DOS kesmeleri tek seferde sadece tek bir ASCII karakter yazdırabilir).
    - **Çözüm:** Sayıyı sürekli 10'a bölüp kalanları Stack'e (yığına) atan ve ardından POP işlemiyle ASCII'ye çevirerek ekrana basan matematiksel bir alt program (`print_num`) geliştirilmiştir.

```
print_num proc
    push ax
    push bx
    push cx
    push dx

    mov cx, 0
    mov bx, 10

bolme_dongusu:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne bolme_dongusu

yazdirma_dongusu:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop yazdirma_dongusu

    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_num endp
```

Resim 5: *Çok haneli sayıları ASCII karakterlerine çevirerek ekrana basan `print_num` alt programının işleyişi.*

6. SONUÇ VE DEĞERLENDİRME

Bu proje, yüksek seviyeli dillerde arka planda çalışan zamanlama, girdi yönetimi ve değişken dönüşümü gibi kavramların donanım seviyesinde nasıl çalıştığının anlaşılamasını sağlamıştır. Projede istenen tüm kısıtlar başarıyla uygulanmış ve "Refleks Oyunu" stabil bir şekilde çalışır hale getirilmiştir.

Not: Projenin derlenebilir `.asm` koduna ve kaynak dosyalarına yukarıdaki GitHub bağlantısı üzerinden erişilebilir.